

NATURAL LANGUAGE PROCESSING · WEEK 2

Text Preprocessing

From raw, noisy text to clean, model-ready tokens

Tokenization · Lowercasing · Punctuation · Stopwords

Stemming · Lemmatization · Contractions · Normalization

Instructor: Muhammad Ehsan

SESSION ROADMAP

What we will cover today

- | | | |
|-----------|-------------------------------|---|
| 01 | Foundations | What preprocessing is, why it matters, where it sits in the NLP pipeline, and the toolkit |
| 02 | Splitting the text | Sentence and word tokenization — plus the five cases where naive tokenizers break |
| 03 | Cleaning the tokens | Lowercasing, punctuation removal, stopword removal → Activity 1 |
| 04 | Normalizing word forms | Stemming vs lemmatization, stemmer comparison → Activity 2 |
| 05 | Extra cleaning steps | Contractions, numbers, special characters, standardization, spelling, OOV, regex |
| 06 | The complete pipeline | One function end-to-end, order of operations, pitfalls → Activity 3 + assignment |

What is text preprocessing?

Text preprocessing is the process of converting raw, unstructured text into a clean, structured format that is suitable for analysis and machine learning.

RAW TEXT (what the world gives you)

" The 2 SUSPECTS weren't caught!!! Visit <http://news.pk/x> "



PREPROCESSED (what the model gets)

['suspect', 'not', 'caught']

1 Remove noise

Punctuation, URLs, HTML, extra whitespace, symbols that carry no signal for the task

2 Standardize language

One casing, one spelling, one word form — so 'Running', 'runs' and 'ran' are not three features

3 Extract components

Break text into the units the model consumes: sentences, words, sub-words, tokens

Key point Garbage in, garbage out. No model recovers from text you never cleaned.

Why does it matter? Importance of Preprocessing

1 Real text is noisy

Typos, casing, punctuation, emojis, HTML tags, double spaces, slang, contractions — none of it is signal.

2 Vocabulary explosion

'Apple', 'apple', 'APPLE', 'apple!' are four different features to a model. Cleaning collapses them into one.

3 Lower dimensionality

A smaller, denser vocabulary means fewer parameters, faster training and less overfitting on small datasets.

4 Consistency

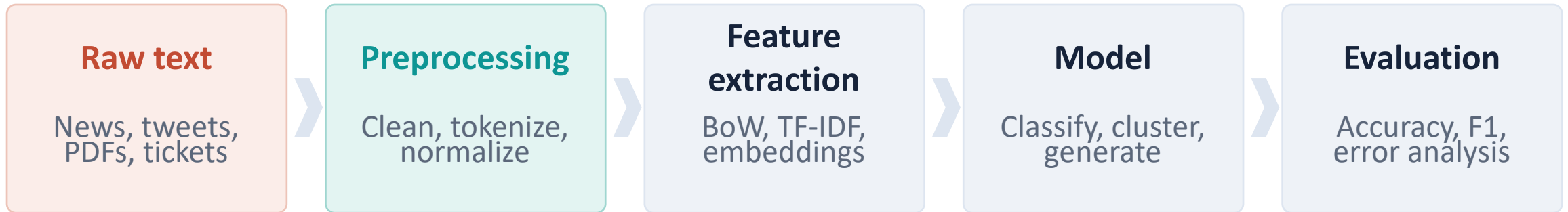
Removes inconsistencies that confuse the algorithm and make results irreproducible across data batches.

A concrete cost of skipping it

Vocabulary of 5 sample sentences without cleaning: **63 unique tokens**
Same 5 sentences after lowercasing, punctuation + stopword removal, lemmatization:
38 unique tokens (a 40% reduction — with no loss of meaning)

Key point Preprocessing is not decoration — it directly changes the size and quality of your feature space.

Where preprocessing sits in the NLP pipeline



Preprocessing is the second stage — and the only one that touches every project, in every language, for every task.

It feeds the next stage directly

- TF-IDF and Bag-of-Words count tokens — so tokens decide the columns
- Word2Vec / GloVe learn from token sequences
- Classical ML gains the most; deep models gain less (see slide 33)

It is task-dependent, not fixed

- Topic modelling: aggressive cleaning helps
- Sentiment analysis: keep negations, keep '!' and emojis
- Named entity recognition: keep casing — it is the signal

The toolkit: libraries you will use

NLTK

Natural Language Toolkit

Tokenization, stemming, lemmatization, stopword lists. The teaching standard — explicit and easy to inspect.

```
nltk.word_tokenize(text)
```

spaCy

Industrial-strength NLP

Fast, production-ready. Tokenizer, lemmatizer, POS tagger and NER in one pass over the document.

```
nlp(text)[0].lemma_
```

re

Regular expressions

Built in. Pattern-based cleaning: strip URLs, emails, hashtags, digits, repeated whitespace.

```
re.sub(r'http\S+', '', text)
```

string

Python standard library

string.punctuation gives you all 32 ASCII punctuation characters as a ready-made filter.

```
string.punctuation
```

TextBlob

Simple wrapper API

One-line lemmatization, spelling correction and sentiment. Great for prototyping and demos.

```
TextBlob(text).correct()
```

gensim

Topic modelling toolkit

Also ships preprocessing utilities: remove_stopwords, simple_preprocess, phrase detection.

```
remove_stopwords(text)
```

Our running example: a counter-terrorism news dataset

Every step today is demonstrated on these five sentences, so you can trace one row all the way from raw text to final tokens.

1 A bombing occurred in downtown Baghdad. Resulting in several casualties and injuries.

2 The terrorist group claimed responsibility for the attack on the military checkpoint.

3 Security forces arrested suspects linked to a plot targeting public transportation.

4 The use of improvised explosive devices has increased in recent insurgent activities.

5 Counter-terrorism units conducted a raid on a suspected militant hideout in the region.

LOADING THE DATA

```
import pandas as pd

data = {
    'ID': [1, 2, 3, 4, 5],
    'Text': [ ... ]
}
df = pd.DataFrame(data)

pd.set_option(
    'display.max_colwidth', None)

df.to_csv('ct_dataset.csv',
          index=False)
```

Notice row 1 already contains a trap: two sentences in one cell, and the abbreviation-free period is a real sentence boundary.

Key point Always look at 5–10 raw rows before writing a single line of cleaning code.



Splitting the text

Tokenization — turning a string into the units a model can count

STEP 1 · TOKENIZATION

Breaking text into tokens

Tokenization divides text into smaller units called tokens. A token may be a sentence, a word, a sub-word or a single character — the choice depends on your model.

Sentence tokenization

Splits a paragraph into sentences.

Input: "A bombing occurred in downtown Baghdad. Resulting in several casualties."

Output: 2 sentences

```
sent_tokenize(text) · doc.sents
```

Word tokenization

Splits a sentence into words and punctuation marks.

Input: "I love NLP!"

Output: ["I", "love", "NLP", "!"]

```
word_tokenize(text) | [t.text for t in doc]
```

Character tokenization

Splits text into single characters.
Used for morphologically rich languages and OCR repair.

Input: "Hello" → H, e, l, l, o

```
list(text)
```

*Sub-word tokenization (**Byte Pair Encoding (BPE)**, *WordPiece*, *SentencePiece*) is what BERT and GPT actually use — it keeps the vocabulary small while still representing unseen words.*

Key point Tokenize first — almost every later step operates on a list of tokens, not on a string.

STEP 1 · TOKENIZATION

Example: tokenizing our dataset

BEFORE — one raw string

"A bombing occurred in downtown Baghdad.
Resulting in several casualties and injuries."



AFTER sent_tokenize — a list of 2 sentences

```
['A bombing occurred in downtown  
Baghdad.',  
 'Resulting in several casualties and  
injuries.']
```

BEFORE — one raw string

"A bombing occurred in downtown Baghdad.
Resulting in several casualties and injuries."



AFTER word_tokenize — a list of 14 tokens

```
['A', 'bombing', 'occurred', 'in',  
 'downtown', 'Baghdad', '.',  
 'Resulting', 'in', 'several',  
 'casualties', 'and', 'injuries', '.']
```

CODE

```
from nltk.tokenize import word_tokenize,  
sent_tokenize  
nltk.download('punkt')  
  
df['sentences'] = df['Text'].apply(sent_tokenize)  
df['tokens'] = df['Text'].apply(word_tokenize)
```

Read the output carefully

- '.' became its own token — punctuation is not gone yet
- 'A' and 'in' survived — stopwords are not gone yet
- Casing is untouched — 'Baghdad' vs 'bombing'

Key point Tokenization only splits. It does not clean — that is the job of the next three steps.

STEP 1 · TOKENIZATION

Where naive tokenizers break (1 of 2)

1 Periods and abbreviations

"Dr. Smith lives in the U.S.A."

Naive: ["Dr", ".", "Smith", ..., "U", ".", "s",
".", "A", "."]

Rule based: ["Dr.", "Smith", "lives", "in",
"the", "U.S.A."]

Fix: Use a trained tokenizer (Punkt, spaCy) that knows abbreviation lists — periods are ambiguous between abbreviation and sentence end.

2 Contractions

"He's going, isn't he?"

Naive: ["He", "'s", "going", ",", "isn", "'t",
"he", "?"]

Rule based: ["He", "is", "going", ",", "is", "not",
"he", "?"]

Fix: Expand contractions BEFORE tokenizing — otherwise 'isn' becomes a meaningless token and the negation is lost.

These two cases alone account for most tokenization bugs in real English corpora.

STEP 1 · TOKENIZATION

Where naive tokenizers break (2 of 2)

3 Hyphenated words

"I bought a high-speed car."

Naive: ["I", "bought", "a", "high", "-", "speed", "car", "."]

Fix: 'high-speed' is one compound concept; splitting it destroys the meaning. But 'well-known' vs 're-schedule' behave differently — context decides.

4 URLs and emails

"Email john.doe@example.com or visit https://example.com."

Naive: ["john", ".", "doe", "@", "example", ".", "com"]

Fix: Strip or mask URLs and emails with a regex before tokenizing: `re.sub(r'http\S+|\S+@\S+', ' ', text)`

5 Special characters and emojis

"Call me @ 5 pm & bring food 🍕"

'@', '&' and emoji dropped or mangled

Fix: On social media these carry sentiment. Use TweetTokenizer or an emoji-aware tokenizer instead of discarding them.

Key point Choose the tokenizer that matches your text domain: news ≠ tweets ≠ clinical notes ≠ code.

02

Cleaning the tokens

Lowercasing · Removing punctuation · Removing stopwords

STEP 2 · LOWERCASING

Collapsing case variation

Converts every character to lowercase so that case-only variants of the same word become a single feature.

BEFORE

```
['Apple', 'apple', 'APPLE', 'ApPle']  
→ 4 distinct features
```



AFTER

```
['apple', 'apple', 'apple', 'apple']  
→ 1 feature
```

BEFORE — from our dataset

```
['Security', 'forces', 'arrested', 'suspects',  
 'Counter-terrorism', 'units']
```



AFTER

```
['security', 'forces', 'arrested', 'suspects',  
 'counter-terrorism', 'units']
```

CODE

```
def lowercase_tokens(tokens):  
    return [w.lower() for w in tokens]  
  
df['lower'] = df['tokens'].apply(lowercase_tokens)
```

When lowercasing hurts

- Named entities: 'Apple' the company vs 'apple' the fruit
- Acronyms: 'US' the country vs 'us' the pronoun; 'IT' vs 'it'
- Emphasis in social text: 'AMAZING' carries more sentiment

Key point Default to lowercasing for classification and topic modelling — but never for NER.

STEP 3 · REMOVING PUNCTUATION

Dropping non-informative symbols

Removes commas, periods, exclamation marks, quotes and other symbols that usually carry no signal for bag-of-words style models.

BEFORE

"Hello, world! How are you?"



AFTER

"Hello world How are you"

BEFORE — tokens from row 1

```
['A', 'bombing', 'occurred', 'in', 'downtown',  
'Baghdad', '.', 'Resulting', ..., 'injuries',  
'...']
```



AFTER — 2 punctuation tokens dropped

```
['A', 'bombing', 'occurred', 'in', 'downtown',  
'Baghdad', 'Resulting', ..., 'injuries']
```

CODE — TWO EQUIVALENT WAYS

```
import string, re  
  
def remove_punctuation(tokens):  
    return [w for w in tokens  
            if w not in string.punctuation]
```

When punctuation matters

- "Let's eat, Grandma!" vs "Let's eat Grandma!" — the comma is the meaning
- '!!!' and '?' are strong sentiment features
- Decimals, prices and code break without their symbols

Key point `string.punctuation` holds 32 ASCII symbols — it will not catch smart quotes “ ” or the em dash —.

STEP 4 · STOPWORD REMOVAL

Dropping high-frequency, low-information words

Stopwords are extremely common words that appear in almost every document and therefore separate documents poorly: is, am, are, the, in, on, and, or, a, of.

BEFORE

"This is a good movie" (5 words)



AFTER

"good movie" (2 words — 60% smaller)

BEFORE — row 1 after punctuation removal (13 tokens)

```
['A', 'bombing', 'occurred', 'in', 'downtown',  
'Baghdad',  
'Resulting', 'in', 'several', 'casualties', 'and',  
'injuries']
```



AFTER — stopwords gone (8 tokens)

```
['bombing', 'occurred', 'downtown', 'Baghdad',  
'Resulting', 'several', 'casualties', 'injuries']
```

CODE

```
from nltk.corpus import stopwords  
stop_words = set(stopwords.words('english'))  
  
def remove_stopwords(tokens):  
    return [w for w in tokens  
            if w.lower() not in stop_words]
```

Danger: NLTK's list contains 'not', 'no', 'nor': USE CUSTOMIZE LIST

"The movie was not good" → "movie good"
Sentiment has been inverted. Always customise the list:

```
stop_words -= {'not', 'no', 'nor', 'never'}
```

Key point NLTK English has 179 stopwords. Inspect the list before you trust it — and always add domain stopwords of your own.

Clean it by hand — no code, 10 minutes

Work in pairs. Apply Steps 1–4 in order and write down the token list after every step.

YOUR SENTENCE

"The Security Forces were NOT able to stop the attack on Tuesday, and 3 suspects escaped!"

1 Word tokenize

Split it by hand. How many tokens do you get? Where does the comma go? Where does '!' go?

2 Lowercase

Which tokens actually change? Does anything become ambiguous once you lowercase it?

3 Remove punctuation

Which characters disappear? Does the sentence lose any meaning?

4 Remove stopwords

Use NLTK's list. Careful — 'not' is on it. Do you keep it or drop it, and why?

Discuss with the room

- How many tokens did you start with, and how many are left?
- Did removing 'not' change what the sentence means? Would you keep the digit '3'?
- Compare with the pair next to you — did you get identical lists? Why not?

Then verify in code

```
word_tokenize(sentence)
→ lowercase → punctuation
→ stopwords
```

Does Python agree with you?

03

Normalizing word forms

Stemming and lemmatization — mapping variants onto one root

STEP 5 · STEMMING

Chopping words down to a root

Stemming applies rule-based suffix stripping (-ing, -ed, -es, -ly, -s) to reduce a word to its stem. It is fast, crude, and the result is often not a real dictionary word.

BEFORE

running · runs · ran · runner



AFTER

run · run · ran · runner

BEFORE — cleaned tokens from our dataset

```
['bombing', 'occurred', 'casualties', 'injuries',  
'activities', 'improvised', 'universities']
```



AFTER PorterStemmer

```
['bomb', 'occur', 'casualti', 'injuri',  
'activ', 'improvis', 'univers']
```

CODE

```
from nltk.stem import PorterStemmer  
stemmer = PorterStemmer()  
  
def stem_text(tokens):  
    return [stemmer.stem(w) for w in tokens]
```

Pros

- Very fast
- Shrinks vocabulary a lot
- No dictionary needed

Cons

- Produces non-words
- Over-stemming: careful → care
- Under-stemming: ran ≠ run

Key point Over-stemming merges words that differ in meaning; under-stemming misses words that should merge. Both are errors.

STEP 5 · STEMMING

Which stemmer should you choose?

Word	Porter	Snowball	Lancaster	Comment
running	run	run	run	All three agree
generously	gener	generous	gen	Lancaster destroys it; Snowball is best
universities	univers	univers	univers	Merges with 'universe' — over-stemming
careful	care	care	car	Meaning lost by all three
maximum	maximum	maximum	maxim	Lancaster is too aggressive
organization	organ	organ	org	'organ' ≠ 'organization'

Porter

The classic. Moderate, well understood, the safe default for English.

Snowball

Porter2 — improved rules, supports 15+ languages. Prefer this over Porter today.

Lancaster

Fastest and most aggressive. Use only when you want maximum vocabulary reduction and can accept damage.

Regex-based

Write your own rules when you know exactly how domain terms must be transformed.

Key point Start with Snowball. Only switch to Lancaster if vocabulary size is the binding constraint.

STEP 6 • LEMMATIZATION

Reducing to a real dictionary word

Lemmatization maps a word to its lemma — its dictionary base form — using a vocabulary (WordNet) plus morphological analysis and the word's part of speech.

BEFORE — cleaned tokens from our dataset

```
['casualties', 'injuries', 'forces',  
'suspects',  
'devices', 'activities', 'units']
```



AFTER WordNetLemmatizer — all real words

```
['casualty', 'injury', 'force', 'suspect',  
'device', 'activity', 'unit']
```

CODE

```
from nltk.stem import WordNetLemmatizer  
nltk.download('wordnet'); nltk.download('omw-1.4')  
lemmatizer = WordNetLemmatizer()  
def lemmatize_text(tokens):  
    return [lemmatizer.lemmatize(w) for w in tokens]
```

The POS trap — the mistake everyone makes

lemmatize('running')	→ 'running'	✗
lemmatize('running', pos='v')	→ 'run'	✓
lemmatize('better', pos='a')	→ 'good'	✓

WordNet defaults to pos='n' (noun). Without POS tagging, most verbs pass through unchanged.

Pros

- Always yields a valid word — output stays readable
- Context-aware, so far more accurate than stemming

Cons

- Slower — a dictionary lookup per token, plus POS tagging
- Needs language-specific resources (WordNet, spaCy models)

STEPS 5 & 6

Stemming vs Lemmatization — side by side

	Stemming	Lemmatization
Method	Rule-based suffix chopping	Dictionary lookup + morphology + POS
Output	A stem — often not a real word	A lemma — always a valid dictionary word
Speed	Very fast	Slower (3–10× depending on tooling)
Accuracy	Approximate	High
Resources	None — just rules	WordNet / spaCy model / language data
'studies'	studi	study
'better'	better	good (with pos='a') adjective
'was'	wa	be (with pos='v') verb
Use it for	Search indexing, topic models, huge corpora, speed-critical jobs	Chatbots, semantic search, anything a human will read

Key point Both collapse variants onto one root. Stemming optimises for speed; lemmatization optimises for correctness.

Predict the output — 8 minutes

Individually, write your prediction for each cell. Then we run the code and score ourselves.

Input word	PorterStemmer output?	WordNetLemmatizer output?	Which one is more useful here?
studies			
studying			
better			
bombing			
casualties			
was			

Questions to answer after you run it

- Which words did the lemmatizer leave completely unchanged, and why?
- Fix those by passing the right pos= argument. What changes?
- For a search engine over news articles, which output would you index — and why?

Starter code

```
words = ['studies', 'studying',  
        'better', 'bombing',  
        'casualties', 'was']  
  
for w in words:  
    print(w, ps.stem(w),  
          lem.lemmatize(w))
```

04

Extra cleaning steps

Contractions · Numbers · Special characters · Standardization · Spelling · OOV

STEP 7 · EXPANDING CONTRACTIONS

Recovering the words hidden inside apostrophes

Expands shortened forms into their full words. Do this BEFORE tokenization, or the tokenizer will produce fragments like 'isn' and "'t".

BEFORE

can't →
don't →
He's going, isn't he?
they've weren't we'll



AFTER

cannot
do not
He is going, is not he?
they have were not we will

CODE

```
import contractions

text = "They weren't caught, can't confirm."
fixed = contractions.fix(text)
# 'They were not caught, cannot confirm.'
```

Why it matters more than it looks

- "weren't" hides the negation 'not' — critical for sentiment
- Makes tokens match the stopword list, which stores full forms
- Unifies "don't" / "dont" / "do not" into one representation

Cost Expansion increases text length, and ambiguous cases are guessed: "He's" can be "He is" or "He has"; "'d" can be "would" or "had". The library picks the most frequent reading.

Key point Order matters: expand contractions → then tokenize → then everything else.

STEPS 8 & 9

Removing numbers and special characters

Removing numbers

Deletes digits when they carry no information for the task.

"I have 2 apples"

"I have apples"

Careful: '2020', '9/11', '1st place' and '3 suspects' are all meaningful. Consider replacing digits with a <NUM> token instead of deleting them.

Removing special characters

Deletes non-alphanumeric symbols: @ # \$ % ^ & * ~ | and emojis

"The price is \$100."

"The price is 100"

Careful: '#Karachi' is a topic label, '@user' is an entity, '\$' changes '100' into a price, and emojis are the strongest sentiment signal on social media.

REGEX RECIPES — RUN THESE ON THE RAW STRING, BEFORE TOKENIZING

```
text = re.sub(r'\d+', '', text) # remove all digits
text = re.sub(r'\d+', '<NUM>', text) # ... or replace them with a placeholder
text = re.sub(r'^[A-Za-z\s]','', text) # keep letters and whitespace only
text = re.sub(r'http\S+|www\.\S+', '', text) # strip URLs
text = re.sub(r'\s+', ' ', text).strip() # collapse repeated whitespace
```

Key point Deletion is irreversible. When in doubt, replace with a placeholder token instead of removing.

STEPS 10 & 11

Normalization and text standardization

Normalization

Converts text into one consistent character-level format — typographic quotes, dashes, accents, Unicode forms.

```
don't → don't  
"quoted" → "quoted"  
café → cafe  
full-width ABC → ABC
```

```
unicodedata.normalize('NFKD', text)
```

Text standardization

Unifies different surface forms of the same concept into one canonical term. Needs domain knowledge.

```
U.S.A. / US / United States → USA  
IED / improvised explosive device → IED  
gonna / gunna → going to  
Pak / PK / Pakistan → Pakistan
```

Implement as a dictionary lookup you maintain per project.

A DOMAIN MAPPING FOR OUR COUNTER-TERRORISM DATASET

```
MAP = {'ied': 'improvised_explosive_device',  
       'counter-terrorism': 'counterterrorism',  
       'u.s.a.': 'usa', 'us': 'usa'}  
  
tokens = [MAP.get(w, w) for w in tokens]
```

The trade-off

Standardization reduces variability, but a wrong mapping silently corrupts every document. Map 'US' → 'USA' and you also rewrite the pronoun 'us'. Always lowercase-and-check in context.

Key point Normalization is generic and safe. Standardization is domain-specific and risky — document every rule.

STEPS 12 & 13

Spelling correction and out-of-vocabulary words

Spelling correction

"I love programing on the compter"

"I love programming on the computer"

`TextBlob(text).correct()` · `SymSpell` · `pyspellchecker`

- Slow — it is an edit-distance search per token
- Will 'correct' rare names and technical terms into the wrong word

Out-of-vocabulary (OOV) words

"The quokka is a friendly animal."

"The [UNK] is a friendly animal."

Three strategies

- Ignore — drop the token entirely (loses information)
- Map to a single [UNK] token (standard for classical models)
- Sub-word tokenization — BPE never has an OOV word at all

Why modern models rarely need either step

BERT-style tokenizers split unknown words into known pieces — 'quokka' becomes 'quo' + '##kka'. The model never sees an [UNK], and misspellings degrade gracefully instead of vanishing. If you are fine-tuning a transformer, spelling correction and OOV handling are usually unnecessary work.

Key point Spelling correction is expensive and risky — apply it only when your text is genuinely noisy (OCR, chat, forms).

TOOLKIT

Regex patterns you will reuse in every project

Goal	Pattern	Before → After
Strip URLs	<code>r'http\S+ www\.\S+'</code>	"visit https://bbc.com now" → "visit now"
Strip emails	<code>r'\S+@\S+\.\S+'</code>	"mail ali@nu.edu.pk" → "mail "
Strip HTML tags	<code>r'<.*?>'</code>	" Breaking news " → "Breaking news"
Keep letters only	<code>r'^[A-Za-z\s]'</code>	"3 IEDs found!" → "IEDs found"
Collapse whitespace	<code>r'\s+' → ' '</code>	"too many\n\nspaces" → "too many spaces"
Remove @mentions	<code>r'@\w+'</code>	"thanks @nustofficial" → "thanks "
Keep hashtag word	<code>r'#(\w+)' → r'\1'</code>	"#Karachi blast" → "Karachi blast"
Squash repeats	<code>r'(\.){1,2}' → r'\1\1'</code>	"soooooo baaad" → "soo baad"

APPLY THEM IN ONE PASS

```
def regex_clean(text):
    for pattern, repl in [(r'<.*?>', ' '), (r'http\S+|www\.\S+', ' '), (r'\S+@\S+\.\S+', ' '), (r'\s+', ' ')]:
        text = re.sub(pattern, repl, text)
    return text.strip()
```

PUTTING IT TOGETHER

The complete pipeline in one function

PREPROCESS.PY

```
import re, string, nltk, contractions
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

stop_words = set(stopwords.words('english'))
stop_words -= {'not', 'no', 'nor', 'never'} # keep negation
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    text = contractions.fix(text) # 1 expand
    text = re.sub(r'http\S+|<.*?>', ' ', text) # 2 strip noise
    text = re.sub(r'\s+', ' ', text).strip()
    tokens = word_tokenize(text) # 3 tokenize
    tokens = [w.lower() for w in tokens] # 4 lowercase
    tokens = [w for w in tokens
               if w not in string.punctuation] # 5 punctuation
    tokens = [w for w in tokens
               if w not in stop_words] # 6 stopwords
    tokens = [lemmatizer.lemmatize(w, pos='v')]
```

Read it as seven ordered decisions

1

Expand contractions

before tokenizing

2

Regex clean

URLs, tags, whitespace

3

Tokenize

string → list

4

Lowercase

collapse case

5

Drop punctuation

symbol tokens out

6

Drop stopwords

negation preserved

7

Lemmatize

one root per word

Key point Wrap the whole pipeline in one function so it is applied identically to train, validation and test data.

```
df['clean_text'] = df['Text'].apply(preprocess)
```

RESULT

Our five sentences, end to end

#	BEFORE — raw text	AFTER — clean_text
1	A bombing occurred in downtown Baghdad. Resulting in several casualties and injuries.	bombing occur downtown baghdad result several casualty injury
2	The terrorist group claimed responsibility for the attack on the military checkpoint.	terrorist group claim responsibility attack military checkpoint
3	Security forces arrested suspects linked to a plot targeting public transportation.	security force arrest suspect link plot target public transportation
4	The use of improvised explosive devices has increased in recent insurgent activities.	use improvise explosive device increase recent insurgent activity
5	Counter-terrorism units conducted a raid on a suspected militant hideout in the region.	counter-terrorism unit conduct raid suspect militant hideout region
63 unique tokens before		38 unique tokens after
40%		vocabulary reduction
0		meaning lost

Note how 'occurred → occur', 'casualties → casualty' and 'forces → force' now share a root with any other form of the same word.

PRACTICE

Order of operations, and five pitfalls

A safe default order

expand contractions → regex clean → tokenize → lowercase → remove punctuation → remove stopwords → lemmatize (or stem) → join

Why this order: contractions must be expanded while apostrophes still exist; stopwords lists store lowercase forms, so lowercase first; lemmatize last, on the smallest token set, because it is the slowest step.

Always sanity-check after each step

```
print(df.loc[0, 'tokens'])  
print(len(set(all_tokens)))  
df['clean_text'].str.len().describe()
```

If a row becomes an empty string after cleaning, your pipeline is too aggressive for that document — find it early, not at model training time.

Five pitfalls that will cost you marks

1

Removing stopwords before lowercasing

'The' is not in the list — only 'the' is. Half your stopwords survive.

2

Stemming and then lemmatizing

The lemmatizer receives non-words like 'casualti' and can do nothing with them.

3

Cleaning train and test differently

Silent accuracy loss that no error message will ever show you.

4

Dropping 'not' with the default stopwords list

Inverts sentiment on every negated sentence in your corpus.

5

Lemmatizing without a POS tag

Verbs pass through untouched — you get most of the cost and little of the benefit.

Key point Inspect one full row after every single step. It takes ten seconds and catches almost every bug.

JUDGEMENT

When you should NOT preprocess

Every step you apply throws information away. Sometimes that information is exactly what the task needs.

Named entity recognition

Keep casing and punctuation

'Apple' vs 'apple' and sentence boundaries are the model's main features.

Sentiment analysis

Keep negations, '!', emojis

'not good' must survive. '!!!' and 😊 carry more signal than most adjectives.

Transformer fine-tuning

Keep almost everything

BERT and GPT tokenizers were pretrained on natural text; heavy cleaning creates a train/test mismatch.

Machine translation

Keep the full sentence

Word order, articles and punctuation are all part of the output you must generate.

Authorship attribution

Keep stopwords above all

Function-word frequency is the strongest stylometric signal there is.

Code or log analysis

Keep symbols and digits

Punctuation is syntax; numbers are identifiers. Removing them destroys the data.

Key point Choose steps from the downstream task backwards — never apply a fixed recipe out of habit.

REFERENCE

Summary of every technique

Step	What it does	Example	Tools
Tokenization	Split text into sentences / words	"I love NLP!" → ['I','love','NLP','!']	NLTK, spaCy
Lowercasing	Collapse case variants	'Apple','APPLE' → 'apple'	Python built-in
Remove punctuation	Delete symbol tokens	"Hello, world!" → "Hello world"	string, re
Stopword removal	Drop uninformative words	"this is a good movie" → "good movie"	NLTK, spaCy, gensim
Stemming	Rule-based root, may be a non-word	'studies' → 'studi'	Porter, Snowball, Lancaster
Lemmatization	Dictionary base form	'studies' → 'study'	WordNet, spaCy, TextBlob
Expand contractions	Restore full words	"can't" → "cannot"	contractions
Remove numbers	Delete or mask digits	"2 apples" → "apples"	re
Remove special chars	Delete @ # \$ % symbols	"\$100" → "100"	re
Normalization	One character-level format	"don't" → "don't"	unicodedata, re
Standardization	One canonical term	'U.S.A.' → 'USA'	custom dictionary
Spelling correction	Fix typos	'programing' → 'programming'	TextBlob, SymSpell
OOV handling	Map unknown words	'quokka' → '[UNK]'	BPE, WordPiece

Build the pipeline — 25-minute lab

Open a fresh notebook. You are given deliberately messy text; your job is to produce clean, model-ready output and justify every choice.

YOUR INPUT DATA

```
docs = [  
    "BREAKING!! 3 suspects weren't caught — police say it's NOT over. More: https://news.pk/a1",  
    "<p>Security forces arrested 2 men in Lahore, Pakistan.</p> #Karachi @dawn_com",  
    "Dr. Khan said the U.S.A. report on IEDs is soooo unclear... he'd re-check it by 5 p.m."  
]
```

A Build it

Write one `preprocess()` function that applies at least seven steps in a defensible order. It must take a raw string and return a clean string.

B Prove it

Print the token list after every single step for document 1. Show the count dropping. Screenshot the before/after.

C Defend it

In markdown, justify three decisions: did you keep 'not'? did you keep '3' and '2'? did you keep '#Karachi'? Stemming or lemmatization — and why?

Stretch goals, if you finish early

- Run the same docs through `PorterStemmer` and `WordNetLemmatizer` — compare vocabularies
- Compute unique-token count before and after; report the percentage reduction

Marking

Working function 40% · Step-by-step evidence 30%
Justification of choices 30%

HOMEWORK

Take-home assignment

Task

Take any public text dataset of at least 500 rows (news headlines, product reviews, tweets — your choice) and produce a preprocessing notebook.

- Show five raw rows, then five cleaned rows, side by side
- Apply and document at least eight preprocessing steps
- Report unique-token count before and after, with the percentage reduction
- Compare stemming vs lemmatization on your own vocabulary — 10 words in a table
- Write a 200-word justification: which steps did you skip, and why does your task not need them?

Deliverables

- One .ipynb with outputs saved
- The cleaned dataset as .csv
- A short README.md

Due: next lab session

Submit as a single .zip

*Notebooks that run top-to-bottom without errors score higher.
Restart the kernel and Run All before you submit.*

Where to find data

Kaggle datasets · Hugging Face Hub (datasets library) · NLTK corpora (reuters, movie_reviews, gutenber) · scikit-learn's fetch_20newsgroups · any public news RSS feed you scrape yourself

Key point Pick a dataset in a domain you care about — you will reuse this cleaning code for your final project.

KEY TAKEAWAYS

What to remember from today

1 Preprocessing is a decision, not a recipe

Every step you apply deletes information. Choose steps by working backwards from the downstream task.

2 Tokenize first, clean second

Almost every technique operates on a list of tokens. Get the split right and the rest follows.

3 Stemming is fast and crude; lemmatization is slow and correct

Use stemming for search and topic models, lemmatization for anything semantic or human-facing.

4 Order matters

Expand contractions before tokenizing, lowercase before stopword removal, lemmatize last.

5 Customise the stopword list

'not', 'no' and 'never' are on NLTK's default list. Removing them will invert your sentiment results.

6 One function, applied everywhere

Train, validation, test and production must all pass through identical cleaning code.

Questions?

Next session: feature extraction — Bag of Words, TF-IDF and word embeddings.

Bring your cleaned dataset; we will vectorise it together.

Reference notebooks: `NLP_Text_Preprocessing.ipynb` · `NLP_Regular_expressions.ipynb`